Контест 3 · asyncio и протоколы

Неделя 6 · темы недель 5–6: повторение T1, корутины и event loop, WebSocket, протокол сообщений, комнаты. Сдача устная у экрана: показываете код, запускаете на примерах, объясняете идею. В задачах B время тоже проверяется: конкурентный код должен работать заметно быстрее последовательного.

Монеты: обычная задача — **2**, со звёздочкой (★, например B4*) — **4**, каждая часть босса — **6**. В этом контесте: 15 обычных, 3 со звёздочкой, 3 частей босса — всего **60** монет. Задача засчитывается, только если вы можете объяснить свой код.

A. Протоколы

A1. Дан лог — список строк, каждая должна быть JSON-объектом с полем `type`. Посчитайте сообщения каждого типа. Всё, что не JSON, не объект или без строкового `type`, считайте под ключом `"invalid"`. Ключи отсортируйте.

```
Ввод:  count_types(['{"type": "move", "col": 3}', '{"type": "ping"}', 'oops', '[1]',
'{"type": 5}'])
Вывод: {"invalid": 3, "move": 1, "ping": 1}
```

A2. Сообщения разделены символом `\n`, но по сети приходят кусками как попало. По списку кусков верните целые сообщения; пустые пропустите, незаконченный хвост отбросьте.

```
Ввод:  split_stream(['{"a":', '1}\n{"b"', ':2}\n{"c"'])
Вывод: ['{"a":1}', '{"b":2}']
```

A3. Кадр — 2 байта длины (big-endian) и текст в UTF-8. Длина считается в **байтах**, а не в символах. Напишите `encode_frame(text) -> bytes` (длиннее 65535 байт — `ValueError`) и `decode_frames(data) -> list[str]` (неполный последний кадр отбросить). Подсказка: `int.to_bytes`, `int.from_bytes`.

```
Ввод:  encode_frame("Ая")
Вывод: b'\x00\x04\xd0\x90\xd1\x8f'

Ввод:  decode_frames(encode_frame("ход 3") + encode_frame("ок") + b"\x00\x05ab")
Вывод: ["ход 3", "ок"]
```

B. asyncio

Во всех задачах дан «медленный сервис» — используйте его, не меняя:

```
async def slow_double(x: int) -> int:
    await asyncio.sleep(0.1)
    return x * 2
```

B1. `async def double_all(xs)` возвращает `[slow_double(x) for x in xs]` в исходном порядке. Для 10 чисел — меньше 0,5 с (последовательно было бы 1 с).

```
Ввод:  asyncio.run(double_all([1, 2, 3]))
Вывод: [2, 4, 6]          # за ~0.1 с
```

B2. Игрок `name` «отвечает» через `delays[name]` секунд. Верните имя первого ответившего и **отмените** остальные задачи, не дожидаясь их. Подсказка: `asyncio.wait(...,`

`return_when=asyncio.FIRST_COMPLETED)`.

```
Ввод:  asyncio.run(first_answer({"Аня": 0.3, "Боря": 0.05, "Вика": 1.0}))
Вывод: "Боря"             # за ~0.05 с
```

B3. `async def with_timeout(coro, seconds, default=None)` возвращает результат корутины или `default`, если она не уложилась в `seconds`.

```
Ввод:  asyncio.run(with_timeout(asyncio.sleep(1, result=1), 0.05, default="таймаут"))
Вывод: "таймаут"
```

B4*. `async def limited(factories, limit)`: `factories` — функции без аргументов, каждая возвращает корутину. Выполните все, но одновременно не больше `limit` (так не перегружают внешний API). Результаты — в исходном порядке. Подсказка: `asyncio.Semaphore`.

```
Ввод:  asyncio.run(limited([lambda i=i: slow_double(i) for i in range(9)], 3))
Вывод: [0, 2, 4, 6, 8, 10, 12, 14, 16]    # за ~0.3 с, одновременно не больше трёх
```

C. Комнаты

C1. Рассадите игроков по порядку прихода в комнаты по `size` человек. Неполная последняя комната — это очередь ожидания. Верните пару (комнаты, ожидающие).

```
Ввод:  assign_rooms(["a", "b", "c", "d", "e"], 2)
Вывод: ([["a", "b"], ["c", "d"]], ["e"])
```

C2. События `(время, "in" | "out")` — входы и выходы игроков. Найдите максимум одновременно онлайн. Если в одну секунду кто-то вышел и кто-то вошёл, сначала обрабатываем выход.

```
Ввод:  max_online([(1, "in"), (2, "in"), (3, "out"), (3, "in"), (5, "in")])
Вывод: 3
```

C3. Игроки ходят по кругу в порядке `players`. По списку того, кто ходил, верните индекс первого хода не в свою очередь или `-1`.

```
Ввод:  first_out_of_turn(["Аня", "Боря", "Аня", "Аня"], ["Аня", "Боря"])
Вывод: 3
```

C4*. Матчмейкинг. Очередь — пары `(имя, рейтинг)`. Идём по очереди: берём первого свободного и ищем ему **первого свободного дальше по очереди** с разницей рейтинга не больше `max_diff`. Нашли — пара, не нашли — игрок ждёт.

```
Ввод:  matchmake([("Аня", 1200), ("Боря", 1500), ("Вика", 1250), ("Гоша", 1480), ("Даня",
900)], 100)
Вывод: [("Аня", "Вика"), ("Боря", "Гоша")]
```

D. WebSocket

Все эндпоинты — в одном FastAPI-приложении. Проверка — через `TestClient` или две вкладки браузера.

D1. `/ws/echo`: на каждое сообщение отвечает `{"type": "echo", "data": <то, что пришло>}`. Если пришёл не JSON — `{"type": "error", "code": "bad_json"}`, соединение не закрываем.

```
Ввод:  {"a": [1, 2]}
Вывод: {"type": "echo", "data": {"a": [1, 2]}}
```

D2. `/ws/counter`: общий на всех счётчик. При подключении клиент получает `{"type": "count", "value": N}`. Любой клиент присылает `{"type": "inc"}` — **все** подключённые получают новое значение. Отключившимся не шлём.

D3. `/ws/chat/{room}?name=Аня`: чат по комнатам. Новый клиент сначала получает `{"type": "history", "messages": [...]}` — последние 5 сообщений комнаты (`{"from", "text"}`). Каждое непустое сообщение рассылается всем в этой комнате как `{"type": "chat", "from": ..., "text": ...}`. Подсказка: `deque(maxlen=5)`.

E. Игра по сети

E1. Запустите сервер из лабы 06 и сыграйте партию с соседом из двух браузеров (или из бота). Покажите в DevTools вкладку Network → WS: сообщения `start`, `state`, `game_over`.

E2. Докажите, что `time.sleep` блокирует сервер. Сделайте эндпоинт `async def`, который спит 2 с через `time.sleep`, и такой же через `asyncio.sleep`. Отправьте 5 запросов одновременно и покажите время: ~10 с против ~2 с. Объясните разницу.

E3. Протокол realtime вашей команды в `SPEC.md` (таблица сообщений и кодов ошибок) и не меньше пяти контрактных тестов, написанных агентом. Покажите, какой тест агента вы исправили или удалили на ревью и почему.

E4 · босс. «Морской бой» на поле 10×10, клетки `(строка, столбец)`. За каждую часть свой плюс.

а) Класс `Fleet`, метод `place(r, c, length, horizontal)` ставит корабль от клетки `(r, c)` вправо или вниз. Если корабль выходит за поле или касается другого (даже углом) — `ValueError`.

б) `shoot(r, c)` возвращает `"miss"`, `"hit"` или `"sunk"` (последняя клетка корабля). Повторный выстрел в ту же клетку или выстрел за поле — `ShotError`. `all_sunk()` — все ли потоплены. `view()` — то, что видит соперник: `.` не стреляли, `o` мимо, `x` ранен, `#` потоплен.

в) Класс `Battle(fleet_a, fleet_b)`, метод `fire(player, r, c)`, игроки 0 и 1. После попадания стреляет тот же игрок, после промаха ход переходит. Последний корабль соперника потоплен — `"win"`. Не в свой ход или после конца игры — `ShotError`.

```
Ввод:  f = Fleet(); f.place(0, 0, 3, True)
       f.shoot(0, 0); f.shoot(1, 1)
       f.view()[:2]
Вывод: ["x.........", ".o........"]
```

E5*. Бот для «Морского боя»: `next_shot(view)` по полю из `view()` выбирает клетку. Если есть раненый корабль (`x`), добиваем: одно ранение — соседи в порядке вверх, вправо, вниз, влево; два и больше в линию — только концы этой линии. Иначе — первая свободная клетка «шахматки» `(r + c) % 2 == 0` (по строкам), а если таких нет — первая свободная.

```
Ввод:  next_shot(<пустое поле, ранения в (4, 4) и (4, 5)>)
Вывод: (4, 3)
```